

昆明理工大学人工智能产业学院学生实验报告

(2025 — 2026 学年 第 2 学期)

课程名称： 自然语言处理 指导教师：陈玮 实验项目名称：综合报告

开课实验室：信自 444 上机时间：2026 年 6 月 13 日

No.	学号	姓名	分 工 内 容	小组评分 (40 分)	教师评分 (60 分)	综合得分 (100 分)
1	202310114106	张旺	独立完成			
2						
3						
4						
5						

教师 评 语	1. 实验原理掌握情况：	A. 了解并掌握 ☐	B. 基本了解 ☐	C. 不了解 ☐
	2. 程序编写和调试能力：	A. 强 ☐	B. 中等 ☐	C. 差 ☐
	3. 实验结果是否达到要求：	A. 达到 ☐	B. 基本达到 ☐	C. 未达到 ☐
	4. 实验过程记录与结果分析情况：			
		A. 记录完整，分析详实 ☐	B. 记录完整，分析不充分 ☐	
		C. 记录不完整，分析不充分 ☐	D. 记录不完整，没有分析 ☐	
	5. 实验报告内容完整性：			
		A. 完整 ☐	B. 基本完整 ☐	C. 不完整 ☐
	6. 报告正文（格式、排版、图表）规范性：			
		A. 规范 ☐	B. 基本规范 ☐	C. 不规范 ☐
	7. 实验资料提交情况：			
		A. 完整 ☐	B. 基本完整 ☐	C. 不完整 ☐
教师签名：				

目 录

实验一 基于前馈神经网络模型实现中文分词	1
一、实验目的及要求	1
二、实验原理分析	1
2.1 中文分词与序列标注	1
2.2 前馈神经网络	2
2.2.1 嵌入层 (Embedding Layer)	2
2.2.2 全连接层与激活函数	2
2.2.3 Dropout 正则化	3
2.2.4 交叉熵损失函数	3
2.3 前馈网络用于分词的局限性	3
三、程序设计流程框图	3
3.1 数据预处理阶段	3
3.2 模型构建阶段	5
3.3 预测解码阶段	6
四、实验过程记录与实验结果分析	6
4.1 训练过程	6
4.2 测试集评估	7
4.3 分词示例分析	7
4.3.1 交互测试	8
4.4 深入分析：为什么 F1 只有 0.5	9

五、结论与体会	9
实验二 词向量表示以及句子相似度计算	11
一、实验目的及要求	11
二、实验原理分析	11
2.1 词的分布式表示	11
2.2 Skip-Gram 模型	11
2.3 负采样技术	12
2.4 余弦相似度与句子向量	12
三、程序设计流程框图	13
3.1 数据预处理	13
3.2 训练数据生成	13
3.3 负采样实现	13
3.4 Skip-Gram 模型	14
3.5 句子向量与相似度计算	14
四、实验过程记录与实验结果分析	15
4.1 训练过程	15
4.2 词向量示例	15
4.3 句子相似度	16
4.4 词相似度	17
4.5 相似词检索	17
4.6 深入分析：数据量对词向量质量的影响	18

五、结论与体会	18
实验三 基于卷积神经网络/循环神经网络实现文本分类任务	19
一、实验目的及要求	19
二、实验原理分析	19
2.1 LSTM 门控机制	19
2.1.1 遗忘门	19
2.1.2 输入门	19
2.1.3 细胞状态更新	20
2.1.4 输出门	20
2.2 双向 LSTM	20
2.3 Attention 机制	20
2.4 TextCNN	21
三、程序设计流程框图	21
3.1 数据集构建	21
3.2 BiLSTM+Attention 模型	22
3.3 TextCNN 模型	22
3.4 训练策略	23
四、实验过程记录与实验结果分析	24
4.1 训练过程对比	24
4.2 测试集整体评估	25
4.3 各类别详细分析	25

4.4 预测示例分析	26
4.5 CNN vs RNN: 为什么 CNN 在这个任务上更强	27
五、结论与体会	28
参考文献	29

实验一 基于前馈神经网络模型实现中文分词

一、实验目的及要求

本实验旨在掌握前馈神经网络模型的原理和结构，并将其应用于中文分词任务。通过本次实验，深入理解中文分词模型训练的一般步骤。

- (1) 掌握前馈神经网络模型的原理和结构，理解各层的作用；
- (2) 掌握中文分词模型训练的一般步骤，包括数据预处理、模型构建、训练和评估；
- (3) 使用 Python 语言实现前馈神经网络，并在通用数据集上训练生成中文分词模型；
- (4) 实现中文分词任务，分析实验结果，理解前馈网络的局限性。

二、实验原理分析

2.1 中文分词与序列标注

中文分词的传统方法包括词典最大匹配和 HMM/CRF 等统计模型。近年来深度学习成为主流，其思路是将分词转化为序列标注任务——为每个字符分配标签，标签表示该字符在词中的位置，据此即可还原分词结果。

BIEOS 用四个标签描述字符在词中的位置，直观清晰：

表 1-1 BIEOS 标签定义及说明

标签	英文全称	含义	示例
B	Begin	多字词的首字符	“北京”中的“北”
I	Inside	多字词的中间字符	“天安门”中的“安”
E	End	多字词的末字符	“北京”中的“京”
S	Single	单字成词	“我”、“爱”

标注规则较为简单：单字词标注 S，多字词首字符标 B、末字符标 E、中间字符标 I。以“我爱北京天安门”为例，假设标准分词为“我/爱/北京天安门”，则标注为：我-S，爱-S，北-B，京-I，天-I，安-I，门-E。遇到 S 或 E 即表示一个词的结束，该信息对后续解码至关重要。

解码是标注的逆过程：沿标签序列从左向右扫描，B-I-...-E 连接为一个词，每个 S 独立成词。该方法简单，但前提是标签序列必须合法，例如 B 后面不能直接跟 S。前馈网络独立预测每个位置的标签，无法保证这种合法性，后文将详细讨论该问题。

2.2 前馈神经网络

前馈神经网络结构最为基础，数据从输入经隐藏层到输出逐层前向传播，无反馈连接也无跳跃连接。本实验采用单隐藏层结构，公式如下：

$$h = \text{ReLU}(W_1 \cdot x + b_1)$$

$$o = W_2 \cdot h + b_2$$

x 为输入的字符嵌入向量， W_1 和 W_2 分别为两个层的权重， b_1 和 b_2 为偏置，ReLU 即 $\max(0, x)$ 。以下逐一介绍各层的作用：

2.2.1 嵌入层 (Embedding Layer)

嵌入层本质上是一张查找表。字符在计算机中仅为整数编号，例如“我”的 ID 可能为 5，“你”可能为 8，这种离散的数字无法参与矩阵运算。嵌入层的作用是为每个 ID 分配一个 d 维实数向量（本实验中 $d=64$ ），训练时这些向量随模型一同更新，逐步编码字符的语义信息。参数量等于词表大小乘以维度， 2768×64 约 17 万参数。

2.2.2 全连接层与激活函数

全连接层将 64 维的嵌入向量先映射到 256 维再映射到 4 维，维度变化为 64-256-4。仅做线性变换显然不够，两层线性叠加本质上仍是线性变换，因此中间引入 ReLU 激活。ReLU 得以广泛应用主要基于两点：一是计算效率高，仅需比较正负；二是正区间梯度恒为 1，反向传播时不会逐层衰减，而 sigmoid 和 tanh 的梯度会逐层衰减，网络加深后便难以训练。

2.2.3 Dropout 正则化

Dropout 值得注意：训练时随机将 30% 的神经元输出置零。初看似乎是在干扰训练，但实际上它迫使网络不依赖单一神经元——某个神经元随时可能被丢弃，网络必须学会通过其他路径完成任务。测试时不再丢弃神经元，全部正常输出，这相当于同时集成了多个子网络。

2.2.4 交叉熵损失函数

分词需区分 4 类标签，采用交叉熵衡量预测与真实标签的差距。公式如下：

$$L = - \sum_i y_i \log(\hat{y}_i)$$

y_i 为真实标签的 one-hot 向量， y_i_hat 为 softmax 后的概率。损失值越小表明预测越准确。PyTorch 的 CrossEntropyLoss 使用便捷，它将 softmax 和交叉熵合并计算，直接传入原始 logits 即可，无需先计算 softmax 再计算损失。

2.3 前馈网络用于分词的局限性

前馈网络存在一个根本缺陷：为每个字符分配标签时仅依据该字符自身，无法感知上下文信息。然而中文分词恰恰依赖上下文——同一个“学”字，在“学生会”中为 B（词首），在“大学生”中为 E（词尾），在“数学”中亦为 E，在“学习”中又为 B。前馈网络无法区分这些情况，只能为所有“学”字给出统一的预测，必然产生大量错误。

此外，BIEOS 标签之间存在严格约束——B 后面只能接 I 或 E，S/E 后面只能接 B 或 S。但前馈网络独立预测，经常产生 B-S、E-I 等非法组合，解码出的分词边界严重混乱。解决该问题需引入 CRF 层约束标签转移，或采用 BiLSTM 建模上下文。

三、程序设计流程框图

本实验的整体流程分为四个阶段：数据预处理-模型构建与训练-预测解码-评估分析。

3.1 数据预处理阶段

(1) 从 MSR 训练文件读取已分词的句子，每行一条，词语之间用空格分隔；

(2) 对每条句子调用 `gen_tags` 方法，生成 BIEOS 标签序列，同时去掉空格拼接回原始文本；

(3) 遍历所有文本中的字符，构建字符到 ID 的映射表 `c2i`，包含 `<PAD>(0)` 和 `<UNK>(1)` 两个特殊标记；

(4) 将文本和标签分别转换为 ID 序列，用于模型训练。

```
class Prep:
    def __init__(self):
        self.c2i = {"<PAD>": 0, "<UNK>": 1}
        self.t2i = {"B": 0, "I": 1, "E": 2, "S": 3}
        self.i2t = {v: k for k, v in self.t2i.items()}

    def gen_tags(self, sent):
        tags = []
        words = sent.strip().split()
        for w in words:
            if len(w) == 1:
                tags.append("S")
            else:
                tags.append("B")
                tags.extend(["I"] * (len(w) - 2))
                tags.append("E")
        return tags, "".join(words)

    def build_vocab(self, texts):
        for t in texts:
            for c in t:
                if c not in self.c2i:
                    self.c2i[c] = len(self.c2i)

    def text2id(self, text):
        return [self.c2i.get(c, 1) for c in text]

    def tag2id(self, tags):
        return [self.t2i[t] for t in tags]
```

表 1-2 数据预处理统计信息

统计项	数值
训练样本数	2000
词汇表大小（含 PAD/UNK）	2768
UNK 率	0.00%
标签类别数	4（B/I/E/S）

3.2 模型构建阶段

模型采用 Embedding + 全连接层的简单结构，具体为：Embedding(2768, 64) - Linear(64, 256) - ReLU - Dropout(0.3) - Linear(256, 4)。各层的输入输出维度变化如下表所示：

表 1-3 模型各层维度变化

层级	输入维度	输出维度	参数量	说明
Embedding	[batch, seq_len]	[batch, seq_len, 64]	2768x64=177,152	字符向量表示
Linear1	[batch, seq_len, 64]	[batch, seq_len, 256]	64x256+256=16,640	特征提取
ReLU	[batch, seq_len, 256]	[batch, seq_len, 256]	0	非线性激活
Dropout	[batch, seq_len, 256]	[batch, seq_len, 256]	0	正则化 (p=0.3)
Linear2	[batch, seq_len, 256]	[batch, seq_len, 4]	256x4+4=1,028	分类输出

```
class Model(nn.Module):
    def __init__(self, vocab, embed, hidden, num_tag):
        super().__init__()
        self.emb = nn.Embedding(vocab, embed, padding_idx=0)
        self.fc1 = nn.Linear(embed, hidden)
        self.relu = nn.ReLU()
        self.drop = nn.Dropout(0.3)
        self.fc2 = nn.Linear(hidden, num_tag)

    def forward(self, x):
        x = self.emb(x)
        x = self.fc1(x)
        x = self.relu(x)
```

```
x = self.drop(x)
return self.fc2(x)
```

3.3 预测解码阶段

预测时，将原始文本送入模型得到每个字符的标签预测，然后按 BIEOS 规则还原分词结果：从左到右扫描标签序列，将字符依次拼接到当前词中，遇到 S 或 E 标签时输出当前词并开始新词。

```
def pred(model, text, prep):
    model.eval()
    with torch.no_grad():
        ids = prep.text2id(text)
        x = torch.tensor([ids])
        out = model(x)
        pred_ids = out.argmax(-1)[0].tolist()
        tags = [prep.i2t[i] for i in pred_ids]
    res, word = [], ""
    for c, t in zip(text, tags):
        word += c
        if t in ("S", "E"):
            res.append(word)
            word = ""
    if word: res.append(word)
    return res
```

四、实验过程记录与实验结果分析

4.1 训练过程

采用 Adam 优化器，学习率 1e-3，批次大小设为 1（即逐句输入），训练 20 个 epoch。训练过程中记录了每个 epoch 的损失和字符级准确率：

表 1-4 训练过程记录

Epoch	Loss	Accuracy	说明
1	0.0251	0.5746	初始阶段，模型快速学习
5	0.0210	0.6564	准确率提升放缓
10	0.0202	0.6661	基本稳定
15	0.0199	0.6706	微幅提升

从训练曲线来看，损失从 0.0251 降至 0.0197，准确率从 57.46% 升至 67.11% 后趋于稳定。该 67% 并非训练不充分所致，而是前馈网络的结构性瓶颈——对于有歧义的字符，仅凭单字信息无法正确分类，67% 基本对应语料中“无歧义字符”的比例。

4.2 测试集评估

在 MSR 测试集上抽取 200 条进行评估，采用词级别的 P/R/F1 进行衡量。评估流程遵循 ICWB2 标准：将 testing 目录的未分词文本输入模型预测，再与 gold 目录的标准答案逐词对比。

表 1-5 测试集评估结果

指标	数值	说明
Precision	0.5022	预测正确的词数/预测总词数
Recall	0.5169	预测正确的词数/标准总词数
F1 Score	0.5066	P 和 R 的调和平均

F1 仅为 0.5066，与工业级分词系统 0.95+ 的水平差距显著。但该结果完全在预期之内——前馈网络独立预测每个字符标签，架构上即决定了其无法处理歧义问题。

4.3 分词示例分析

在测试集评估中，选取几个典型示例分析模型在各类歧义上的表现：

表 1-6 测试集分词示例及评估

输入文本	模型输出	标准分词	P	R	F1
扬帆远东做与	扬/帆远东做/	扬帆/远东/做/	0.4286	0.3750	0.4000
中国合作的先	与/中/国合作/	与/中国/合作/			

行	的/先行	的/先行			
希腊的经济结构较特殊	希腊的/经济/结构/较/特殊/。	希腊/的/经济/结构/较/特殊/。	0. 8333	0. 7143	0. 7692
他要与中国人合作	他/要/与/中/国人/合作/。	他/要/与/中国/人/合作/。	0. 7143	0. 8333	0. 7692

“扬帆远东做与中国合作的先行”的 F1 仅为 0.40，是三个示例中最差的。该句存在多处交叉歧义叠加——“扬帆”被拆为“扬/帆远东做”，“中国”被拆为“中/国合作”，模型几乎无法正确处理任何一处歧义。“希腊的经济结构较特殊”和“他要与中国人合作”的 F1 约 0.77，相对较好但仍不理想——“希腊的”被合并、“中国人”被拆分，都是典型的上下文依赖错误。

4.3.1 交互测试

训练完成后进行交互式分词测试，输入任意句子观察模型输出：

表 1-7 交互测试结果

输入	模型输出
我爱北京天安门	我/爱/北京天安门
今天天气很好	今天天气/很/好
我在学习深度学习	我/在学习/深度/学习

交互测试进一步验证了前馈网络的局限性：“北京天安门”被合并为一个大词，模型无法识别这是两个独立地名；“今天天气”中两个“天”相邻但属于不同词，模型统一预测导致边界错误；“深度学习”作为固定搭配被拆分为“深度”和“学习”。归纳而言，错误大致分为几类：（1）交叉歧义——“今天天气”中两个“天”相邻但属于不同词，模型为所有“天”统一预测，边界因此出错；（2）组合歧义——“深度学习”作为固定搭配，却被拆分为“深度”和“学习”两个词；（3）专有名词——“北京天安门”被合并为一个大词，模型缺乏相应知识以识别这是两个独立地名。

一个值得注意的例子是“希腊的”被合并为一个词。训练语料中“的”字作为单字词（S 标签）出现频率很高，但在“希腊的”这一语境下，模型可能将“希”判为 B、“腊”判为 I、“的”判为 E，组合为一个三字词。其本质是前馈网络对“的”字在不同上下文中无法区分——它仅依据“的”字本身，无法感知上下文信息。

4.4 深入分析：为什么 F1 只有 0.5

F1 仅为 0.5，归根结底源于前馈网络的三个结构性缺陷：

一是独立预测缺乏语境。为每个字符分配标签时仅依据该字符本身，无法获取上下文信息。而中文分词恰恰依赖上下文，同一字符在不同位置标签可能完全不同。前馈网络无法处理此类歧义，只能输出“平均最优”标签，边界判断必然产生大量错误。

二是标签一致性无法保证。BIEOS 标签有严格约束：B 后面只能接 I 或 E，S/E 后面只能接 B 或 S。但前馈网络独立预测，B-S、E-I 等非法组合频繁出现，解码出的分词边界严重混乱。CRF 层可通过学习转移矩阵约束标签序列，而前馈网络缺乏此机制。

三是 67% 的训练准确率即为上限。有歧义的字符，无论如何训练，仅凭单字信息都无法正确分类。67% 基本对应语料中“无歧义字符”的比例——标点符号、罕见字等在任何上下文中标签均固定，模型可正确处理；标签随上下文变化的字符，模型只能选择最频繁的标签，准确率自然难以提升。

五、结论与体会

通过本实验，我对前馈网络在序列标注任务上的局限性有了直观的认识。BIEOS 标注确实巧妙，为每个字符分类即可还原分词结果，思路清晰且实现简单。但前馈网络独立预测的机制从根本上无法处理歧义，67% 的训练准确率和 0.5066 的测试 F1 基本是“仅看单字”所能达到的上限。

最典型的例子是“今天天气”——两个“天”相邻，标签却完全不同（前一个 E 后一个 B），前馈网络无法区分。这使我真正理解了序列标注为何需要上下文建模：并非锦上添花，而是不可或缺。后续采用 BiLSTM 捕捉序列信息，或引入 CRF 层约束标签转移，F1 应能显著提升。

此外，ICWB2 的评估流程也值得关注——testing 和 gold 分开存储，评估采用词级 P/R/F1 而非字符级准确率，更能反映分词质量。字符级 67%看似尚可，词级 F1 仅为 0.5，说明一个词中只要有一个字符标注错误则整个词即判定错误，错误的放大效应十分显著。

实验二 词向量表示以及句子相似度计算

一、实验目的及要求

本实验旨在掌握词向量的表示方法和句子相似度计算方法。通过本次实验，理解如何将词语转换为向量表示，并在此基础上实现句子相似度计算。

- (1) 掌握词向量的表示方法，理解分布式表示的核心思想；
- (2) 掌握句子相似度计算方法，理解余弦相似度的原理；
- (3) 使用 Python 语言实现将词语转换为向量表示；
- (4) 在词向量基础上实现两个句子之间的相似度计算。

二、实验原理分析

2.1 词的分布式表示

早期词的表示方法为 one-hot：每个词对应一个超长向量，仅自身位置为 1，其余全为 0。问题显而易见——维度与词表规模一致，达数万至数十万，向量极度稀疏；且任意两个 one-hot 向量均正交，“北京”与“上海”的距离同“北京”与“苹果”完全相同，无法表达语义关系。

分布式表示采用不同思路，以低维稠密向量（通常 50 至 300 维）表示词语，每个维度无明确含义，但整个向量编码了语义信息。其理论基础为 Firth 于 1957 年提出的分布假说：“You shall know a word by the company it keeps”，即一个词的含义由其常与哪些词共现来决定。Word2Vec 即基于该假说，通过预测上下文来学习词向量。

2.2 Skip-Gram 模型

Skip-Gram 是 Word2Vec 两种架构之一（另一种为 CBOW），思路直观：给定中心词，预测其周围窗口内出现的词语。模型包含两套嵌入矩阵——中心词矩阵 W 和上下文词矩阵 W' ，每个词在这两套矩阵中各有一个向量。

给定中心词 w_c 和上下文词 w_o ，Skip-Gram 旨在最大化其共现概率。原始 Softmax 概率公式：

$$p(w_o | w_c) = \frac{\exp(v_o' \cdot v_c)}{\sum_{w \in V} \exp(v_w' \cdot v_c)}$$

v_{w_c} 为中心词向量， v_{w_o} 为上下文词向量，分母需要遍历整个词表 V 。词表规模较大时（例如 10 万个词），计算一次 Softmax 需 10 万次点积，计算代价过高。因此引入了负采样技术。

2.3 负采样技术

负采样的核心思想是将多分类转化为二分类。对每个正样本对 (w_c, w_o) ，随机采样 k 个负样本词 $w_{n1} \dots w_{nk}$ ，训练目标是使正样本点积增大、负样本点积减小。损失函数：

$$L = -\log \sigma(v_o' \cdot v_c) - \sum_{i=1}^K \log \sigma(-v_{n_i}' \cdot v_c)$$

其中 σ 为 sigmoid 函数。如此每次仅需计算 $k+1$ 个词对的点积，复杂度从 $O(|V|)$ 降至 $O(k)$ ， k 通常取 5 至 20，速度提升数千倍。

负样本并非均匀随机采样，采样概率按词频的 0.75 次幂计算：

$$P_n(w) = \frac{f(w)^{0.75}}{\sum_w f(w)^{0.75}}$$

为何采用 0.75 次幂？目的在于抑制高频词、提升低频词的采样概率。举例而言：词 A 频率 1000，词 B 频率 1，均匀采样时 A 被抽中的概率为 B 的 1000 倍；经 0.75 次幂调整后 A 的权重变为 $1000^{0.75}$ 约 178，B 仍为 1，比值从 1000:1 缩小至 178:1。否则“的”“是”等高频功能词将占据绝大部分负样本名额，低频实词几乎难以被有效训练。

2.4 余弦相似度与句子向量

余弦相似度衡量两个向量的夹角，忽略向量幅度仅关注方向，取值范围为 -1 到 1，越接近 1 表示越相似：

$$\cos(A, B) = \frac{A \cdot B}{\|A\| \times \|B\|}$$

句子向量采用最简单的平均池化：将句子中所有词向量求和后除以词数。实现确实简单，但缺陷也很明显——功能词（“的”“是”“了”）与关键词（“自然”“语言”“处理”）权重相同，词序信息也完全丢失。“我 爱 北京”与“北京 爱 我”得到的句子向量完全相同，但语义截然不同。更优的方法包括 TF-IDF 加权、SIF，或直接使用 Sentence-BERT 等预训练编码器。

三、程序设计流程框图

3.1 数据预处理

从 MSR 训练文件读取 500 条已分词语料，每条按空格切词，然后构建词到 ID 的映射表（词表 2807 个词），同时统计每个词的出现频率，用于后续计算负采样概率。

3.2 训练数据生成

采用滑动窗口（大小 2）生成训练数据。对句子中每个词，取前后各 2 个词作为上下文，组成(中心词 ID, 上下文词 ID)对。500 条句子共生成 39712 个训练对。

```
def gen_data(sents, prep, window_size=2):
    data = []
    for sent in sents:
        ids = prep.sent2id(sent)
        for i, center in enumerate(ids):
            start = max(0, i - window_size)
            end = min(len(ids), i + window_size + 1)
            for j in range(start, end):
                if i != j:
                    data.append((center, ids[j]))
    return data
```

3.3 负采样实现

负采样概率按词频 0.75 次幂计算，然后使用 numpy 的 random.choice 按该分布采样。每次采样 k=5 个负样本，确保不与目标词重复。

```
def init_neg_prob(prepare):
    vocab_size = prepare.size()
    word_freq = np.array([prepare.word_counts.get(prepare.i2w[i], 1)
                          for i in range(vocab_size)])
    word_freq = word_freq ** 0.75
    prob = word_freq / word_freq.sum()
    return prob
```

3.4 Skip-Gram 模型

```
class SkipGram(nn.Module):
    def __init__(self, vocab_size, embed_dim):
        super().__init__()
        self.emb_center = nn.Embedding(vocab_size, embed_dim)
        self.emb_context = nn.Embedding(vocab_size, embed_dim)
        self.log_sigmoid = nn.LogSigmoid()

    def forward(self, center, context, negative):
        center_emb = self.emb_center(center)
        context_emb = self.emb_context(context)
        negative_emb = self.emb_context(negative).unsqueeze(0)
        pos_score = torch.sum(center_emb * context_emb, dim=1)
        neg_score = torch.bmm(negative_emb,
                              center_emb.unsqueeze(2)).squeeze(2)
        loss = -self.log_sigmoid(pos_score).mean() \
              - self.log_sigmoid(-neg_score).mean()
        return loss
```

3.5 句子向量与相似度计算

```
def sent_vec(model, ids):
    model.eval()
    with torch.no_grad():
        vecs = model.emb_center(
            torch.tensor(ids, dtype=torch.long))
        if len(vecs) == 0:
            return np.zeros(model.emb_center.embedding_dim)
        return vecs.mean(0).numpy().reshape(1, -1)

def sim(v1, v2):
    return cosine_similarity(v1, v2)[0][0]
```

表 2-1 模型训练参数配置

参数	值	说明
词向量维度	32	较低维度，适合小数据集
窗口大小	2	前后各取 2 个词
负采样数 k	5	每个正样本配 5 个负样本
优化器	SGD	随机梯度下降
学习率	0.05	SGD 需要较大学习率
训练轮数	20	迭代次数
词表大小	2807	500 条语料的词表

四、实验过程记录与实验结果分析

4.1 训练过程

表 2-2 训练损失变化

Epoch	Loss	说明
1	3.0026	初始阶段，损失较高
5	1.6380	下降放缓
10	1.4300	继续下降
15	1.2589	接近收敛
20	1.0896	最终损失

训练完成后，损失从 3.0 降至 1.09，整体呈收敛趋势。但客观来看 1.09 的最终损失偏高，模型仍有优化空间——可能原因包括数据量不足、维度偏低，或训练轮次不够。

4.2 词向量示例

训练完成后观察几个常见词的向量表示（仅展示前 5 维）：

表 2-3 部分词向量示例（前 5 维）

词	dim1	dim2	dim3	dim4	dim5
我	-0.7707	-0.1346	0.4264	0.3941	-0.6776
爱	-0.1569	-0.7274	0.6949	0.8680	-0.7695
北京	-1.3847	-0.8712	-0.2234	1.7174	0.3189

中国	-1.3847	-0.8712	-0.2234	1.7174	0.3189
天气	-1.3847	-0.8712	-0.2234	1.7174	0.3189

最引人注目的现象是“北京”“中国”“天气”三个语义完全不同的词向量竟然完全相同！这显然不合理。深入分析原因：这三个词均为低频词，在 500 条句子的语料中出现的上下文窗口极少，训练过程中几乎未被有效更新。PyTorch 默认 Embedding 初始化采用 $N(0, 1)$ 分布，这些低频词的初始向量恰巧较为接近，有限的梯度更新又将它们拉到了同一个局部最优点。这是小数据集训练词向量的典型问题——低频词无法获得足够的训练信号，向量退化为“死向量”，彼此无法区分。

4.3 句子相似度

表 2-4 句子相似度计算结果

句子 1	句子 2	余弦相似度	合理性判断
我 爱 北京	我 爱 中国	1.0000	偏高，结构相同但语义不同
今天 天气 很好	明天 天气 晴朗	0.9469	合理，主题相同
自然 语言 处理	机器 学习	0.7949	偏高，同属 AI 领域但细分不同
我 爱 北京	自然 语言 处理	0.5595	合理，语义无关
北京 是 中国 的 首都	我 爱 中国	0.7651	偏高，部分相关

句子相似度的排序基本合理——主题相同的句子确实比主题无关的相似度更高。但绝对值普遍偏高，“我 爱 北京”与“我 爱 中国”相似度竟达 1.0，这显然不合理。分析原因有两层：一是平均池化对短句过于敏感，两个 3 词句子若有 2 个词相同（“我”和“爱”），平均向量就会非常接近；二是“北京”“中国”“天气”向量完全相同，进一步推高了相似度——“我 爱 北京”和“我 爱 中国”中唯一不同的词向量竟然一样，平均池化后自然完全重合。

4.4 词相似度

表 2-5 词对相似度计算结果

词对	余弦相似度	预期	评价
北京 与 中国	1.0000	0.3~0.5	严重偏高，向量趋同
天气 与 晴朗	1.0000	0.5~0.7	偏高，有关联但非同义
学习 与 机器	0.1287	0.1~0.3	偏低，共现少
自然 与 语言	0.1929	0.3~0.5	偏低，固定搭配
算法 与 问题	-0.1014	0.1~0.3	负值，训练不充分

4.5 相似词检索

表 2-6 相似词检索结果

目标词	Top-3 相似词及相似度	评价
北京	登记 (0.5868), 噪音 (0.5772), 文 (0.5643)	与“北京”语义无关
中国	登记 (0.5868), 噪音 (0.5772), 文 (0.5643)	与“中国”语义无关，且与北京完全相同
天气	登记 (0.5868), 噪音 (0.5772), 文 (0.5643)	与“天气”语义无关，且与北京完全相同
学习	干 (0.6335), 机会 (0.6234), 地下 (0.5829)	与“学习”语义无关

相似词检索效果不佳——“北京”的相似词为“登记”“噪音”“文”，与语义毫无关联。根本原因在于数据量不足：500 条句子约 4 万个训练对，而 Word2Vec 原论文采用 Google News 的 1 万

亿词语料，相差百万倍。数据如此稀疏，大部分词的共现信息极其有限，模型无法学习到有意义的语义关系。

4.6 深入分析：数据量对词向量质量的影响

本次实验最核心的发现是：词向量质量严重依赖数据量。500 条句子、2807 个词、4 万个训练对——远远不够。具体表现为三个方面：

一是低频词向量趋同。大量低频词在训练中几乎未被更新，向量基本停留在初始化附近。最极端的例子是“北京”“中国”“天气”三个语义完全不同的词向量竟然完全相同——它们的 Top-3 相似词也完全一致（登记、噪音、文），说明这三个词在向量空间中坍塌到了同一个点。PyTorch 默认 Embedding 初始化范围较小，低频词的初始向量恰巧较为接近，有限的梯度更新又将它们拉到了同一个局部最优点，形成“死向量”。

二是相似词结果接近随机。4 万个训练对，平均每个词仅 14 个上下文样本（39712/2807 约 14），不足以捕捉语义关系。Word2Vec 原论文建议至少需要数十亿词的语料才能训练出高质量的词向量。

三是句子相似度偏高。平均池化本身会使相似度偏高（词向量均值方向较为集中），低频词向量趋同进一步加剧了该问题。

五、结论与体会

通过本实验，我对词向量训练有了更为实际的认识。Skip-Gram 思路确实巧妙——通过预测上下文来学习词的语义，将“分布假说”转化为可计算的模型。负采样将计算量从 $O(|V|)$ 降至 $O(k)$ ，使大规模训练具有可行性。0.75 次幂的采样概率设计亦很精妙，既抑制了高频词又保障了低频词的训练机会。

但实验也使我深刻认识到数据量的关键作用。500 条句子训练出的词向量效果不佳——最极端的表现是“北京”“中国”“天气”三个语义完全不同的词向量完全相同，相似词检索接近随机，句子相似度异常偏高（“我爱北京”与“我爱中国”达 1.0），与 Word2Vec 原论文采用万亿级语料训练出的效果差距悬殊。此外，平均池化的句子向量确实过于粗糙，功能词与关键词权重相同，词序信息完全丢失，后续可尝试 TF-IDF 加权或 SIF 方法。

实验三 基于卷积神经网络/循环神经网络实现文本分类任务

一、实验目的及要求

本实验旨在掌握卷积神经网络/循环神经网络的原理和结构，并将其应用于文本分类任务。通过本次实验，理解文本分类任务的一般步骤。

- (1) 掌握卷积神经网络/循环神经网络的原理和结构；
- (2) 掌握文本分类任务的一般步骤；
- (3) 使用 Python 语言实现卷积神经网络和循环神经网络；
- (4) 实现文本分类任务，对比两种模型的性能差异。

二、实验原理分析

2.1 LSTM 门控机制

普通 RNN 存在固有缺陷：处理长序列时梯度随时间步指数级衰减，远距离的依赖关系无法学习。LSTM 专门解决这一问题——它引入细胞状态和三个门控单元，使梯度能够传播更远。

2.1.1 遗忘门

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

遗忘门决定上一时刻的细胞状态 C_{t-1} 需要遗忘哪些信息。sigmoid 输出 0 到 1 之间的值，0 表示完全遗忘，1 表示完全保留。例如读到“他出生在北京，后来去了上海”，读到“上海”时遗忘门应将“北京”相关的信息压低。

2.1.2 输入门

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c)$$

输入门控制新信息的写入。 i_t 控制写入量， C_t_hat 为候选值（ $\tanh$ 压缩至-1 到 1），两者逐元素相乘即为实际写入的内容。

2.1.3 细胞状态更新

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

旧细胞状态经遗忘门筛选，加上经输入门筛选的新值，得到新的细胞状态。该加法操作是 LSTM 缓解梯度消失的关键——梯度可通过 C_{t-1} 到 C_t 的直通路径传播，无需反复乘以小于 1 的权重。

2.1.4 输出门

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

输出门控制细胞状态经 $\tanh$ 压缩后输出为当前隐状态 h_t 的量。 h_t 将传递至下一时刻，同时用于当前时刻的预测。

2.2 双向 LSTM

单向 LSTM 仅能获取左侧历史信息，但许多 NLP 任务需要同时考虑左右两侧上下文。双向 LSTM 的思路直观：两个 LSTM 分别从左向右和从右向左处理序列，每个位置将两个方向的隐状态拼接：

$$h_t = [\vec{h}_t; \overleftarrow{h}_t]$$

如此每个位置便拥有完整的上下文信息。但 BiLSTM 输出为变长序列，需通过池化或 Attention 聚合成固定长度向量后方可送入分类层。

2.3 Attention 机制

Attention 使模型学习应关注序列的哪个部分，而非简单地取最后一个隐状态或全部平均。本实验在 BiLSTM 模型中采用了注意力机制，计算流程如下：

(1) 对 LSTM 每个时刻的输出 h_t 做线性变换加 $\tanh$ 激活，计算“能量”： $e_t = \tanh(W_a * h_t)$

(2) 将能量映射为标量分数： $s_t = v_a * e_t$ ，其中 v_a 为可学习的权重向量

(3) 对所有时刻的分数做 softmax 归一化，得到注意力权重： $a_t = \exp(s_t) / \sum(\exp(s_k))$

(4) 用权重对隐状态加权求和，得到固定长度的上下文向量： $c = \sum(a_t * h_t)$

Attention 的优势在于权重 a_t 是动态计算的，不同输入关注的位置不同；且权重可可视化，有助于理解模型关注的具体位置。

2.4 TextCNN

TextCNN 的思路是将文本分类视为一维卷积任务。核心是使用不同尺寸的卷积核在词嵌入序列上滑动，分别提取不同长度的 n -gram 特征。本实验采用 3 种尺寸（2、3、4），分别对应 bigram、trigram 和 4-gram。

每个卷积核滑过输入序列产生特征图，然后进行全局最大池化，提取最显著的特征值。最大池化的优势在于：关键特征无论出现在序列的哪个位置均可捕获，具有一定的平移不变性。

本实验采用双层 Conv1d 加全局最大池化的结构——每个尺度使用两个 Conv1d 层，中间插入 BatchNorm 和 ReLU，最后对每个尺度的输出做全局最大池化，将三个尺度的池化结果拼接后送入全连接层分类。双层卷积的设计使得第一层提取基本的 n -gram 模式，第二层在第一层特征上进一步组合，能够学习到更抽象的特征。

三、程序设计流程框图

3.1 数据集构建

从 ICWB2 的 msr、pku、cityu 三个训练文件中读取数据，筛选词数在 5 至 25 之间的句子，每个语料库最多取 2000 条（总计 max_lines=6000），每条作为一条样本，标签即为其来源语料库。经筛选后共计约 6000 条，80%训练，20%测试，词表 15744 个词。

分类依据何种特征区分？主要有三点：（1）分词粒度不同——同一短语在不同语料库中切分方式不同，例如“中央政府”在 msr 中可能为一个词，在 pku 中可能拆为“中央/政府”；（2）用词习惯——cityu 含粤语表达（“發生”“於”“該”），msr 和 pku 为大陆简体；（3）繁简体——cityu 使用繁体字，msr 和 pku 使用简体字。

3.2 BiLSTM+Attention 模型

```
class Attention(nn.Module):
    def __init__(self, hidden):
        super().__init__()
        self.attn = nn.Linear(hidden, hidden)
        self.v = nn.Linear(hidden, 1, bias=False)

    def forward(self, hidden_states):
        energy = torch.tanh(self.attn(hidden_states))
        scores = self.v(energy).squeeze(-1)
        weights = torch.softmax(scores, dim=1)
        return (hidden_states * weights.unsqueeze(-1)).sum(dim=1)

class BiLSTM(nn.Module):
    def __init__(self, vocab, embed, hidden, num_class,
                 num_layers=2, dropout=0.3):
        super().__init__()
        self.emb = nn.Embedding(vocab, embed, padding_idx=0)
        self.emb_drop = nn.Dropout(0.2)
        self.lstm = nn.LSTM(embed, hidden, num_layers,
                             batch_first=True, dropout=dropout,
                             bidirectional=True)
        self.attention = Attention(hidden * 2)
        self.fc = nn.Sequential(
            nn.Dropout(0.5), nn.Linear(hidden*2, hidden),
            nn.ReLU(), nn.Dropout(0.3),
            nn.Linear(hidden, num_class))
```

3.3 TextCNN 模型

```
class CNN(nn.Module):
    def __init__(self, vocab, embed, num_class,
                 num_filters=128, filter_sizes=(2,3,4)):
        super().__init__()
        self.emb = nn.Embedding(vocab, embed, padding_idx=0)
        self.emb_drop = nn.Dropout(0.2)
        self.convs = nn.ModuleList([
            nn.Sequential(
                nn.Conv1d(embed, num_filters, fs, padding=fs//2),
                nn.BatchNorm1d(num_filters), nn.ReLU(),
                nn.Conv1d(num_filters, num_filters, fs, padding=fs//2),
                nn.BatchNorm1d(num_filters), nn.ReLU()
            ) for fs in filter_sizes
        ])
        total = len(filter_sizes) * num_filters
        self.fc = nn.Sequential(
            nn.Dropout(0.5), nn.Linear(total, 128),
```

```

        nn.ReLU(), nn.Dropout(0.3),
        nn.Linear(128, num_class))

def forward(self, x):
    x = self.emb(x)
    x = self.emb_drop(x).transpose(1, 2)
    conv_outs = []
    for conv in self.convs:
        c = conv(x)
        c = F.adaptive_max_pool1d(c, 1).squeeze(-1)
        conv_outs.append(c)
    x = torch.cat(conv_outs, dim=1)
    return self.fc(x)

```

3.4 训练策略

表 3-1 训练参数配置

参数	值	说明
优化器	AdamW	带权重衰减的 Adam
学习率	1e-3	初始学习率
学习率调度	CosineAnnealingLR	余弦退火
损失函数	CrossEntropyLoss	带类别权重
梯度裁剪	max_norm=1.0	防止梯度爆炸
Early Stopping	patience=7	监控 Val F1
批次大小	32	mini-batch
最大序列长度	50	截断/填充
训练轮数	30	最大 epoch 数

类别权重平衡旨在应对样本不均衡——三个类别虽各 2000 条，但训练集划分后可能存在轻微偏差。权重公式 $w(c) = N / (C * n(c))$ ， N 为总样本数， C 为类别数， $n(c)$ 为类别 c 的样本数。样本较少的类别权重较大，防止模型偏向多数类。

四、实验过程记录与实验结果分析

4.1 训练过程对比

表 3-2 RNN (BiLSTM+Attention) 训练过程

Epoch	Loss	Acc	Val_Acc	Val_F1
1	0.8851	0.5735	0.6975	0.7108
5	0.2616	0.9023	0.7625	0.7835
10	0.0896	0.9712	0.8150	0.8267
15	0.0432	0.9879	0.8467	0.8509
20	0.0101	0.9962	0.8425	0.8455

RNN 在 epoch 15 达到最佳 Val F1 (0.8509)，此后 Val F1 开始下降，训练 Loss 却继续降低——典型的过拟合信号。Early Stopping 在 epoch 21 触发，Best Val F1 为 0.8569。

表 3-3 CNN (TextCNN) 训练过程

Epoch	Loss	Acc	Val_Acc	Val_F1
1	1.0329	0.4790	0.6508	0.6585
5	0.3933	0.8494	0.8067	0.8218
10	0.1637	0.9427	0.8308	0.8397
15	0.0853	0.9727	0.8550	0.8606
20	0.0438	0.9856	0.8567	0.8613

CNN 的 Val F1 持续上升至 epoch 20 的 0.8613，过拟合程度较轻。Early Stopping 在 epoch 25 触发，Best Val F1 为 0.8692。

从训练过程来看，几个差异较为显著：

（1）RNN 初始收敛更快——第 1 轮 RNN 的 Val_Acc 即达 0.6975（Val F1=0.7108），CNN 的 Val_Acc 仅为 0.6508（Val F1=0.6585）。这是因为 LSTM 对序列模式更为敏感，初期即可捕获序列规律。

（2）但 RNN 更容易过拟合——epoch 15 时 RNN 训练 Loss 仅为 0.0432（Acc=98.79%），Val_Acc=0.8467，Val F1=0.8509；到 epoch 20 时训练 Loss 降至 0.0101（Acc=99.62%），Val_Acc 却降至 0.8425，Val F1 降至 0.8455。CNN 训练 Loss 下降较缓，但 Val_Acc 持续上升至 0.8567，Val F1 达 0.8613，泛化能力确实更好。

（3）Early Stopping 有效防止了过拟合——RNN 在 epoch 21 即停止（Best Val F1=0.8569），CNN 在 epoch 25 停止（Best Val F1=0.8692）。若无早停机制，RNN 的过拟合将更加严重。

4.2 测试集整体评估

表 3-4 两个模型测试集整体性能对比

模型	Accuracy	Macro Precision	Macro Recall	Macro F1
BiLSTM+Attention	0.8250	0.8440	0.8250	0.8344
TextCNN	0.8592	0.8694	0.8592	0.8642

CNN 的 Macro F1（0.8642）比 RNN（0.8344）高 3 个百分点，Accuracy 也高 3.4%。客观而言这出乎预期——直觉上 BiLSTM 能建模长距离依赖应更具优势。但分析各类别表现后，原因便清楚了。

4.3 各类别详细分析

表 3-5 各类别性能对比

类别	模型	Precision	Recall	F1	样本数
----	----	-----------	--------	----	-----

cityu	BiLSTM	0.7342	0.9875	0.8422	400
cityu	TextCNN	0.7932	0.9875	0.8797	400
msr	BiLSTM	0.9049	0.7375	0.8127	400
msr	TextCNN	0.8853	0.8300	0.8568	400
pku	BiLSTM	0.8929	0.7500	0.8152	400
pku	TextCNN	0.9297	0.7600	0.8363	400

按类别分析，几个现象值得关注：

一是 cityu 的 Recall 极高。两个模型在 cityu 上 Recall 均为 0.9875——400 条仅错 5 条。原因在于 cityu 具有粤语用词（“發生”“於”“該”“號稱”）和繁体字等显著特征，识别难度很低。CNN 在 cityu 上 F1（0.8797）比 RNN（0.8422）高，主要因为 Precision 更高——CNN 判断更为谨慎，误判更少。

二是 msr 和 pku 容易混淆。两个模型在这两个类别上的 Recall 均偏低（0.73~0.83），因为两者均为大陆简体中文，用词习惯高度重叠，分词粒度的细微差异难以捕捉。RNN 在 msr 上 Recall 仅为 0.7375，超过 1/4 的 msr 样本被误判为其他类别。

三是 cityu 的 Precision 偏低。RNN 在 cityu 上 Precision 仅为 0.7342，有不少 msr/pku 样本被误判为 cityu。推测原因是部分样本中恰巧出现了粤语特征词（例如“返港”），模型倾向于将其归为 cityu，而该语句可能仅引用了粤语词汇。

4.4 预测示例分析

表 3-6 预测示例及分析

输入文本（截取）	预测	标准	结果	分析
陈 方 安生 返 港 后 表示 中央政府 重视 贯彻...	cityu	pku	✗	“返 港”有港式色彩导致误判

地震 發生 於 昨 日 凌晨 2 時 31 分...	cityu	cityu	✓	繁体字“發生”“於” 是强信号
巴基斯坦 的 拉合 尔市 发生 严重 枪击 事件...	pku	pku	✓	大陆简体风格，判 断正确
到底 如何 看待 鲁迅 所 受 普列 汉诺夫...	cityu	msr	✗	缺乏显著特征，倾 向 cityu
更 妙 的 是 ， 面条 与 筷子 是 天生 的 搭档...	msr	msr	✓	口语化表达，msr 风格
該 項 兩 年 一 度 的 賽 事 號 稱 「 消 防 奧 運 會 」...	cityu	cityu	✓	繁体字+粤语，特 征明显

从预测示例来看，RNN 和 CNN 给出了完全相同的预测结果，这说明两个模型学到的分类特征高度一致。模型主要依据两类特征进行判断：（1）繁体字和粤语用词是 cityu 的强信号，几乎不会判错；（2）msr 和 pku 的区分主要依赖分词粒度的细微差异，模型容易混淆。6 个示例中 4 个正确、2 个错误，错误均发生在缺乏显著区分特征时——“陈 方 安生 返 港”虽为 pku 语料，但“返 港”的港式色彩使模型误判为 cityu；“到底 如何 看待 鲁迅”缺乏任何语料库特征词，模型默认预测 cityu。

4.5 CNN vs RNN：为什么 CNN 在这个任务上更强

CNN 的 Macro F1 比 RNN 高 3 个百分点，核心原因是该任务的分类信号主要为局部特征——粤语用词、繁体字、分词粒度差异，均为 n-gram 级别的模式。CNN 的多尺度卷积核恰好擅长提取此类局部特征：2-gram 提取“返 港”等双字组合，3-gram 提取“中央政府”等三字短语，4-gram 提取更长的固定搭配。信息从输入到分类层路径短，信号衰减少，效率高。

RNN 则需要逐步处理序列，信息需经过多步 LSTM 传递才能到达分类层。BiLSTM 确实能建模长距离依赖，但本任务并不需要——判断文本来源无需理解整段话的含义，提取几个关键局部特征即可。RNN 的长距离建模能力在此未能发挥优势，反而因更容易过拟合而表现更差。

训练过程亦能印证这一点：RNN 第 1 轮 Val F1 即 0.71，LSTM 确实初始收敛快；但后续训练 Loss 下降过快（epoch 15 即 0.04），验证集 F1 反而下降——典型的过拟合。CNN 的训练 Loss 下降较缓，验证集 F1 持续上升，说明 CNN 的归纳偏置与该任务更为匹配。

五、结论与体会

通过本实验，对 CNN 和 RNN 各自的特点有了更为具体的认识。在这个以局部特征为关键信号的文本分类任务上，CNN 确实更为合适——多尺度卷积核直接提取 n-gram 模式，最大池化提取最显著特征，简单高效且不易过拟合。RNN 虽能建模长距离依赖，但本任务并不需要，反而因参数更多、更易过拟合而表现较差。

几个训练策略的效果也很显著：Early Stopping 及时抑制了过拟合（RNN 在 epoch 21 即停止，继续训练只会更差），类别权重平衡使模型不偏向多数类，梯度裁剪防止了 RNN 的梯度爆炸，余弦退火使学习率后期逐步下降有助于收敛。这些技巧与模型结构同等重要，缺少任何一项效果都会受到影响。

总之，选择模型应依据任务特点——局部特征为主则适用 CNN，需要长距离依赖则选用 RNN，也可将两者结合（CNN 提取局部特征再输入 LSTM 建模序列关系）。没有万能模型，只有最合适的。

参考文献

- [1] Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need[C]//Advances in Neural Information Processing Systems. Long Beach: Curran Associates, 2017: 5998–6008.
- [2] Mikolov T, Chen K, Corrado G, et al. Efficient estimation of word representations in vector space[C]//Proceedings of the 1st International Conference on Learning Representations (ICLR). Scottsdale, 2013.
- [3] Mikolov T, Sutskever I, Chen K, et al. Distributed representations of words and phrases and their compositionality[C]//Advances in Neural Information Processing Systems. Lake Tahoe: Curran Associates, 2013: 3111–3119.
- [4] Hochreiter S, Schmidhuber J. Long short-term memory[J]. Neural Computation, 1997, 9(8): 1735–1780.
- [5] Kim Y. Convolutional neural networks for sentence classification[C]//Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP). Doha: Association for Computational Linguistics, 2014: 1746–1751.
- [6] Graves A, Schmidhuber J. Framewise phoneme classification with bidirectional LSTM and other neural network architectures[J]. Neural Networks, 2005, 18(5–6): 602–610.
- [7] Bahdanau D, Cho K, Bengio Y. Neural machine translation by jointly learning to align and translate[C]//Proceedings of the 3rd International Conference on Learning Representations (ICLR). San Diego, 2015.
- [8] Zhang X, Zhao J, LeCun Y. Character-level convolutional networks for text classification[C]//Advances in Neural Information Processing Systems. Montreal: Curran Associates, 2015: 649–657.
- [9] Loshchilov I, Hutter F. Decoupled weight decay regularization[C]//Proceedings of the 7th International Conference on Learning Representations (ICLR). New Orleans, 2019.
- [10] Kingma D P, Ba J. Adam: A method for stochastic optimization[C]//Proceedings of the 3rd International Conference on Learning Representations (ICLR). San Diego, 2015.